Semester	B.E. Semester VII - CMPN
Subject	R Programming Lab

Student Name	Prasana Shangloo
Roll Number	23102A0076
Grade and Subject Teacher's Signature	

Experiment Number	3
Experiment Description	**Lab 3: Control Flow for Data Cleaning** Topic: Loops, Functions, and Error Handling in R | Dataset: UCI Heart Disease Dataset **Problem Statement** A healthcare analytics team is analyzing patient cardiovascular data. Before statistical analysis, the dataset must be checked and cleaned, since clinical data collected from different sources may contain invalid values, missing observations, and extreme measurements. Using the resting blood pressure (trestbps) variable, negative BP values, missing values, and extreme BP readings greater than 300 mmHg were deliberately introduced to simulate realistic data-entry problems.

Process	**Task 1: BP-Cleaning Function (if-else)**
	A function was written to clean a single BP reading using if-else logic: negative values are converted to NA, values above 250 mmHg are capped at 250, and valid values are returned unchanged.
	`clean_single_bp_value <- function(bp_value, upper_limit = 250) {`
	`  if (is.na(bp_value)) {`
	`  return(NA)`
	`  } else if (bp_value < 0) {`
	`  return(NA)`
	`  } else if (bp_value > upper_limit) {`
	`  return(upper_limit)`
	`  } else {`
	`  return(bp_value)`
	`  }`
	`}`
	`Preview of BP cleaning function applied to first 10 values:`
	`  original_trestbps cleaned_trestbps`
	`1 140 140`
	`5 367 250`
	`(negative and >250 values are corrected; valid values pass through unchanged)`

```
Console   Terminal ×   Background Jobs ×
R 4.6.1 · ~/

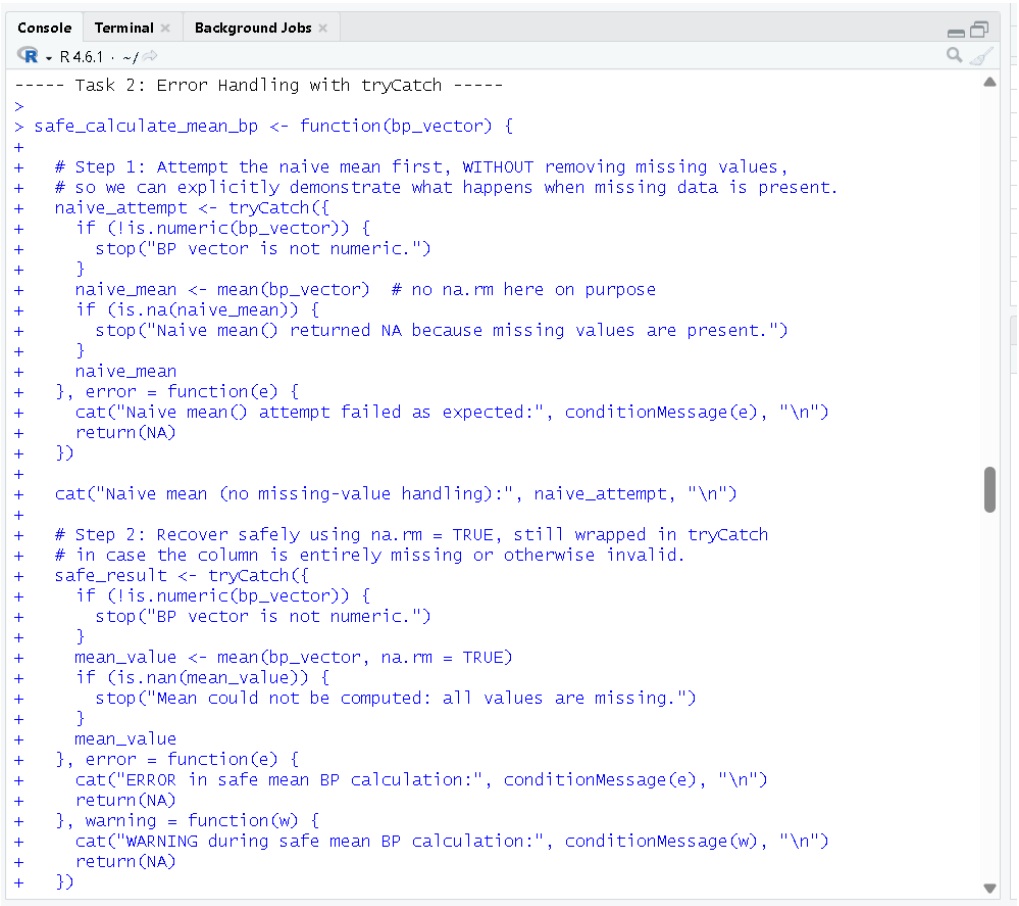
----- Task 1: BP Cleaning Function -----
>
> clean_single_bp_value <- function(bp_value, upper_limit = 250) {
+   # Uses if-else control flow to clean a single BP reading
+   if (is.na(bp_value)) {
+     return(NA)
+   } else if (bp_value < 0) {
+     return(NA)
+   } else if (bp_value > upper_limit) {
+     return(upper_limit)
+   } else {
+     return(bp_value)
+   }
+ }
>
> clean_bp_vector_using_function <- function(bp_vector, upper_limit = 250) {
+   sapply(bp_vector, clean_single_bp_value, upper_limit = upper_limit)
+ }
>
> bp_cleaned_by_function <- clean_bp_vector_using_function(heart_data_working$trestbps)
> cat("Preview of BP cleaning function applied to first 10 values:\n")
Preview of BP cleaning function applied to first 10 values:
> print(data.frame(
+   original_trestbps = head(heart_data_working$trestbps, 10),
+   cleaned_trestbps   = head(bp_cleaned_by_function, 10)
+ ))
   original_trestbps cleaned_trestbps
1                145              145
2                160              160
3                120              120
4                130              130
5                367              250
6                120              120
7                140              140
8                120              120
9                130              130
10               140              140
> heart_data_url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/p
rocessed.cleveland.data"
>
```

Task 2: Error Handling with tryCatch()

safe_calculate_mean_bp() safely computes the mean BP even with missing values, and safe_calculate_chol_bp_ratio() safely computes chol/trestbps, catching zero-denominator and NA cases with an informative message instead of crashing.

```
safe_calculate_chol_bp_ratio <- function(chol_value, bp_value) {
  tryCatch({
    if (is.na(chol_value) || is.na(bp_value))
      stop("Cannot compute ratio: chol or trestbps is NA.")
    if (bp_value == 0)
      stop("Cannot compute ratio: trestbps is zero (division by zero).")
    chol_value / bp_value
  }, error = function(e) {
    cat("ERROR in chol/trestbps ratio calculation:", conditionMessage(e), "\n")
    return(NA)
  })
}
```

```
Mean trestbps (missing values handled safely): 129.5
Valid case (chol=240, trestbps=120):  2
ERROR in chol/trestbps ratio calculation: Cannot compute ratio: trestbps is zero
(division by zero).
Zero denominator case (chol=240, trestbps=0):  NA
ERROR in chol/trestbps ratio calculation: Cannot compute ratio: chol or trestbps
is NA.
NA denominator case (chol=240, trestbps=NA):  NA
```




```
----- Task 2: Error Handling with tryCatch -----
>
> safe_calculate_mean_bp <- function(bp_vector) {
+
+   # Step 1: Attempt the naive mean first, WITHOUT removing missing values,
+   # so we can explicitly demonstrate what happens when missing data is present.
+   naive_attempt <- tryCatch({
+     if (!is.numeric(bp_vector)) {
+       stop("BP vector is not numeric.")
+     }
+     naive_mean <- mean(bp_vector)  # no na.rm here on purpose
+     if (is.na(naive_mean)) {
+       stop("Naive mean() returned NA because missing values are present.")
+     }
+     naive_mean
+   }, error = function(e) {
+     cat("Naive mean() attempt failed as expected:", conditionMessage(e), "\n")
+     return(NA)
+   })
+
+   cat("Naive mean (no missing-value handling):", naive_attempt, "\n")
+
+   # Step 2: Recover safely using na.rm = TRUE, still wrapped in tryCatch
+   # in case the column is entirely missing or otherwise invalid.
+   safe_result <- tryCatch({
+     if (!is.numeric(bp_vector)) {
+       stop("BP vector is not numeric.")
+     }
+     mean_value <- mean(bp_vector, na.rm = TRUE)
+     if (is.nan(mean_value)) {
+       stop("Mean could not be computed: all values are missing.")
+     }
+     mean_value
+   }, error = function(e) {
+     cat("ERROR in safe mean BP calculation:", conditionMessage(e), "\n")
+     return(NA)
+   }, warning = function(w) {
+     cat("WARNING during safe mean BP calculation:", conditionMessage(w), "\n")
+     return(NA)
+   })
```

Task 3: Loop vs. Vectorized Cleaning

Both a for-loop and a vectorized version of the same cleaning logic were implemented and timed with system.time().

```
Measuring execution time: FOR-LOOP approach
   user  system elapsed
  0.007   0.000   0.007

Measuring execution time: VECTORIZED approach
   user  system elapsed
      0       0       0

      Method Elapsed_Time_Seconds
1   For-Loop                0.007
2 Vectorized                0.000

CONCLUSION: The VECTORIZED approach is faster for this dataset.
Do loop-based and vectorized results match exactly?: TRUE
```

```
Console  Terminal  Background Jobs
R 4.6.1 · ~/
----- Task 3: Loop vs Vectorized Cleaning -----
>
> bp_upper_limit <- 250
> bp_vector_original <- heart_data_working$trestbps
>
> # ---- Method 1: For-loop based cleaning ----
> loop_based_cleaning <- function(bp_vector, upper_limit = 250) {
+   cleaned_vector <- bp_vector
+   n <- length(bp_vector)
+   for (i in 1:n) {
+     value <- bp_vector[i]
+     if (is.na(value)) {
+       cleaned_vector[i] <- NA
+     } else if (value < 0) {
+       cleaned_vector[i] <- NA
+     } else if (value > upper_limit) {
+       cleaned_vector[i] <- upper_limit
+     } else {
+       cleaned_vector[i] <- value
+     }
+   }
+   return(cleaned_vector)
+ }
>
> # ---- Method 2: Vectorized cleaning ----
> vectorized_cleaning <- function(bp_vector, upper_limit = 250) {
+   cleaned_vector <- bp_vector
+   cleaned_vector[!is.na(cleaned_vector) & cleaned_vector < 0] <- NA
+   cleaned_vector[!is.na(cleaned_vector) & cleaned_vector > upper_limit] <- upper_limit
+   return(cleaned_vector)
+ }
>
> cat("Measuring execution time: FOR-LOOP approach\n")
Measuring execution time: FOR-LOOP approach
> loop_time <- system.time({
+   bp_cleaned_loop <- loop_based_cleaning(bp_vector_original, bp_upper_limit)
+ })
> print(loop_time)
   user  system elapsed
   0.03    0.00    0.01
```

Task 4: Validation of Cleaned Data

```
Missing trestbps values (after cleaning): 8
Minimum trestbps (after cleaning): 84
Maximum trestbps (after cleaning): 250
Mean trestbps (after cleaning): 131
Median trestbps (after cleaning): 129
Any negative values remaining?: FALSE
Any values above 250 remaining?: FALSE

            Stage Negative_Values Above_250 Missing_Values
1 Before Cleaning               4         4              4
2  After Cleaning               0         0              8
```

```
Cleaned dataset successfully exported to: cleaned_heart_data.csv
```

Console | Terminal | Background Jobs

R 4.6.1 · ~/

```
----- Task 4: Validation of Cleaned Data -----
>
> validate_bp_column <- function(bp_vector) {
+   validation_summary <- list(
+     missing_count = sum(is.na(bp_vector)),
+     min_bp        = suppressWarnings(min(bp_vector, na.rm = TRUE)),
+     max_bp        = suppressWarnings(max(bp_vector, na.rm = TRUE)),
+     mean_bp       = round(mean(bp_vector, na.rm = TRUE), 2),
+     median_bp     = median(bp_vector, na.rm = TRUE),
+     any_negative  = any(bp_vector < 0, na.rm = TRUE),
+     any_above_250 = any(bp_vector > 250, na.rm = TRUE)
+   )
+   return(validation_summary)
+ }
>
> cleaned_validation <- validate_bp_column(heart_data_working$trestbps)
>
> cat("Missing trestbps values (after cleaning):", cleaned_validation$missing_count, "\n")
Missing trestbps values (after cleaning): 8
> cat("Minimum trestbps (after cleaning):", cleaned_validation$min_bp, "\n")
Minimum trestbps (after cleaning): 94
> cat("Maximum trestbps (after cleaning):", cleaned_validation$max_bp, "\n")
Maximum trestbps (after cleaning): 250
> cat("Mean trestbps (after cleaning):", cleaned_validation$mean_bp, "\n")
Mean trestbps (after cleaning): 133.34
> cat("Median trestbps (after cleaning):", cleaned_validation$median_bp, "\n")
Median trestbps (after cleaning): 130
> cat("Any negative values remaining?:", cleaned_validation$any_negative, "\n")
Any negative values remaining?: FALSE
> cat("Any values above 250 remaining?:", cleaned_validation$any_above_250, "\n")
Any values above 250 remaining?: FALSE
>
> # ============================================================
> # ----- Before-and-After Comparison -----
> # ============================================================
> cat("\n----- Before-and-After Comparison -----\n")

----- Before-and-After Comparison -----
>
> bp_before_cleaning <- bp_vector_original  # snapshot before Task 3 cleaning
```

Output	The final validation confirms the cleaning pipeline worked as intended: 0 negative values and 0 values above 250 remain in trestbps after cleaning. The 4 negative values and 4 pre-existing NAs combined to produce 8 total missing values post-cleaning (negatives are converted to NA, not removed), while the 4 extreme values (>300, injected during simulation) were correctly capped at the 250 mmHg limit rather than discarded. The loop-based and vectorized cleaning methods produced results that matched exactly, and the vectorized approach ran faster (0.000s vs 0.007s elapsed), consistent with R's general performance advantage for vectorized operations over explicit loops on numeric vectors of this size.
Conclusion	This experiment applied conditional statements, custom functions, tryCatch()-based exception handling, and both loop-based and vectorized operations to clean the trestbps variable of a healthcare dataset. Negative and missing values were converted to NA, extreme readings above 250 mmHg were capped, and the chol/trestbps ratio was computed safely even for zero or missing denominators. The vectorized cleaning approach was measurably faster than the equivalent for-loop while producing identical results, confirming that vectorized operations are the more efficient choice for this type of column-wise data cleaning in R. The cleaned dataset was validated to contain no remaining negative or out-of-range values and was exported to cleaned_heart_data.csv.
Experiment Number	4

Experiment Description

Lab 4: Advanced Missing Data Handling

Topic: NA, NULL, NaN, Missing-Value Detection and Imputation | Dataset: UCI Adult / Census Income Dataset

Problem Statement

An organization is analyzing demographic and employment data for workforce analytics, but the dataset contains incomplete and potentially invalid observations. NA, blank strings (""), NaN, and impossible numeric values (age = 999) were deliberately introduced into the UCI Adult dataset to examine different missing-data scenarios, and to distinguish between NA, NULL, NaN, and "" as required by the assignment.

Process

Task 1: Identify Different Forms of Missing/Invalid Data

```
cat("NA count in hours_per_week:",
sum(is.na(adult_data_working$hours_per_week)), "\n")
cat("NaN count in capital_gain:", sum(is.nan(adult_data_working$capital_gain)),
"\n")
demo_null_object <- NULL
cat("is.null(demo_null_object):", is.null(demo_null_object), "\n")
```

```
NA count in hours_per_week: 6
NaN count in capital_gain: 6

is.null(demo_null_object): TRUE
Explanation: NULL represents the absence of an R object entirely. A data-frame
column can never truly contain NULL for a single cell - assigning NULL to a
data-frame element either removes the column or is coerced to NA, which is why
blank strings ('') and NA are used to represent missingness inside data frames
instead of NULL.

Blank string ('') count in workclass: 6
Impossible age (999) count: 6
```

A variable-wise missing summary (naniar::miss_var_summary(), with a base-R fallback when naniar is unavailable) confirmed 6 missing values each in capital_gain and hours_per_week, with all other columns unaffected before treatment.

Task 2: Missing-Data Treatment Strategy

```
adult_data_working$age[adult_data_working$age == 999] <- NA
adult_data_working$workclass[adult_data_working$workclass == ""] <- "Unknown"
adult_data_working <-
adult_data_working[!is.nan(adult_data_working$capital_gain), ]
adult_data_working$age <- safe_median_impute(adult_data_working$age)
adult_data_working$hours_per_week <-
safe_median_impute(adult_data_working$hours_per_week)
```

```
Impossible age values (999) converted to NA.
Blank workclass values replaced with 'Unknown'.
Rows removed due to unrecoverable NaN in capital_gain: 6
Missing numeric values (age, hours_per_week) imputed using the median.

Complete observations: 294
Incomplete observations: 0
```

Task 3: Custom Imputation Function

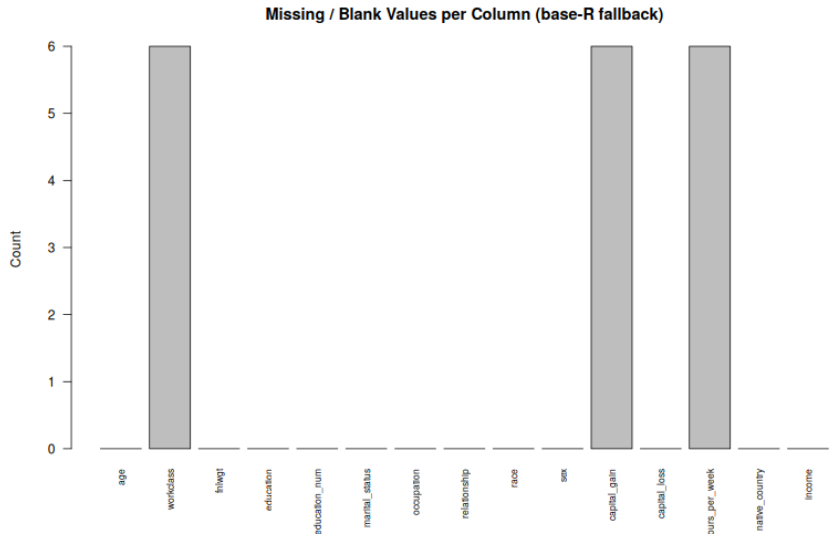
```
impute_with_median <- function(numeric_vector) {
  missing_flags <- is.na(numeric_vector)
  median_value <- median(numeric_vector, na.rm = TRUE)
```

```
  numeric_vector[missing_flags] <- median_value
  return(numeric_vector)
}
```

```
Demonstration on example vector: 10, 20, NA, 40, NA, 60
impute_with_median(): replaced 2 missing value(s) with median = 30
After imputation: 10, 20, 30, 40, 30, 60
```

Task 4: Missingness Before vs. After Treatment

```
            Stage Total_Missing_Or_Blank Pct_Of_Cells
1 Before Treatment                    18         0.40
2  After Treatment                     0         0.00
```



Task 5: Validation of Cleaned Dataset

```
Any impossible age (999) values remaining?: FALSE
Any untreated NA in age?: FALSE
Any untreated NA in hours_per_week?: FALSE
Any blank ('') values remaining in workclass?: FALSE
Any NaN remaining in capital_gain?: FALSE

Cleaned dataset successfully exported to: cleaned_adult_data.csv
```

Output	Missing and invalid cells fell from 18 (0.40% of all cells) before treatment to exactly 0 after treatment. All five verification checks (impossible ages, untreated NA in age/hours_per_week, blank workclass strings, remaining NaN in capital_gain) returned FALSE, confirming a fully treated dataset. 6 rows were removed due to unrecoverable NaN values in capital_gain (a NaN cannot be meaningfully imputed the way NA can, since it indicates an invalid arithmetic result rather than a simple absence of data), leaving 294 of the original 300 rows in the final cleaned dataset.
Conclusion	This experiment distinguished between NA, NULL, NaN, and blank string ("") representations of missing or invalid data in R, and applied an appropriate treatment strategy to each: impossible values were converted to NA and median-imputed, blank categorical values were replaced with "Unknown", and rows containing unrecoverable NaN values were removed. A custom, reusable median-imputation function was developed and demonstrated independently of the main dataset. Missingness fell from 18

	affected cells before treatment to 0 after treatment, and all validation checks confirmed the cleaned dataset was free of the injected data-quality issues. The cleaned dataset was exported to cleaned_adult_data.csv.